

## Data Hazards: Forwarding Unit

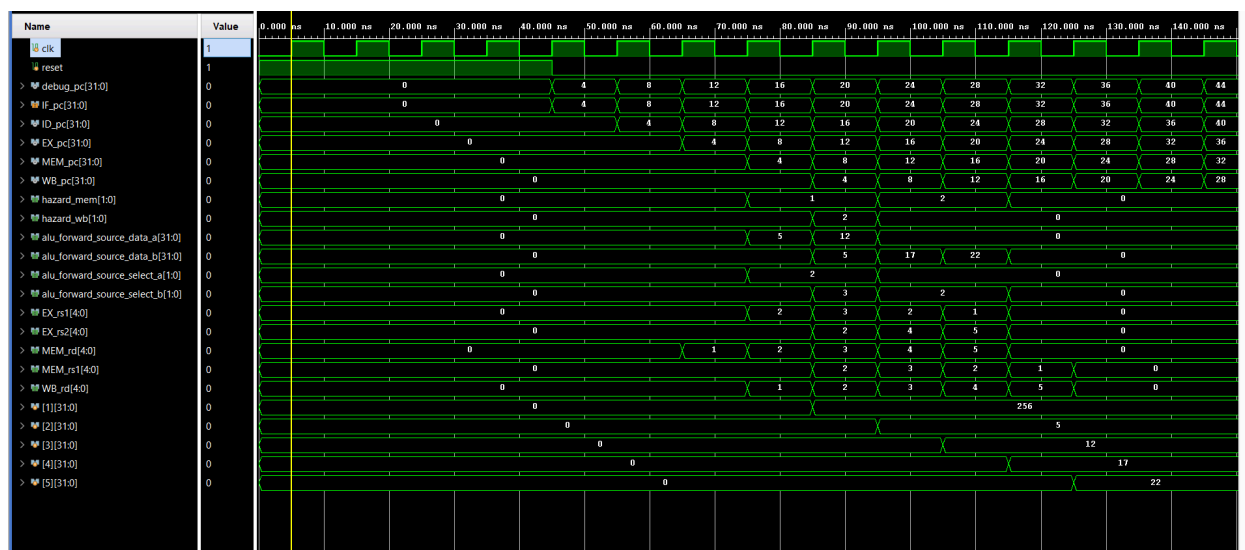
The Forwarding Unit is designed to bypass data from the MEM and WB pipeline stages directly to the ALU inputs, resolving Read-After-Write (RAW) hazards without stalling the processor.

The following sequence was executed to test simultaneous forwarding to both ALU inputs:

```
addi x2, x0, 5    // WB Stage
addi x3, x2, 7    // MEM Stage
add  x4, x3, x2   // EX Stage (Needs x3 from MEM, x2 from WB)
```

| Signal Name     | Binary Value | Description                                          |
|-----------------|--------------|------------------------------------------------------|
| EX_rs1          | 00011        | ALU Input A needs x3.                                |
| EX_rs2          | 00010        | ALU Input B needs x2.                                |
| MEM_rd          | 00011        | Prior instruction is writing to x3 (Matches EX_rs1). |
| WB_rd           | 00010        | Older instruction is writing to x2 (Matches EX_rs2). |
| hazard_mem[1:0] | 01           | Flags an rs1 hazard from the MEM stage.              |

|                             |    |                                                  |
|-----------------------------|----|--------------------------------------------------|
| hazard_wb[1:0]              | 10 | Flags an rs2 hazard from the WB stage.           |
| alu_forward_source_select_a | 10 | Mux A selects data forwarded from the MEM stage. |
| alu_forward_source_select_b | 11 | Mux B selects data forwarded from the WB stage.  |



As shown in the waveform during the clock cycle where the instruction at EX\_pc = 12 enters the Execute stage, the ALU requires operands from registers x3 and x2 (EX\_rs1 = 3, EX\_rs2 = 2), which are concurrently being modified by older instructions in the Memory and Writeback stages (MEM\_rd = 3, WB\_rd = 2). The hardware immediately detects these dependencies (hazard\_mem = 1, hazard\_wb = 2) and updates the multiplexer control lines (alu\_forward\_source\_select\_a = 2, alu\_forward\_source\_select\_b = 3). This successfully forwards the correct values (12 and 5) directly from the MEM and WB pipeline registers into the ALU inputs, allowing the processor to seamlessly execute the instruction without inserting a stall bubble.

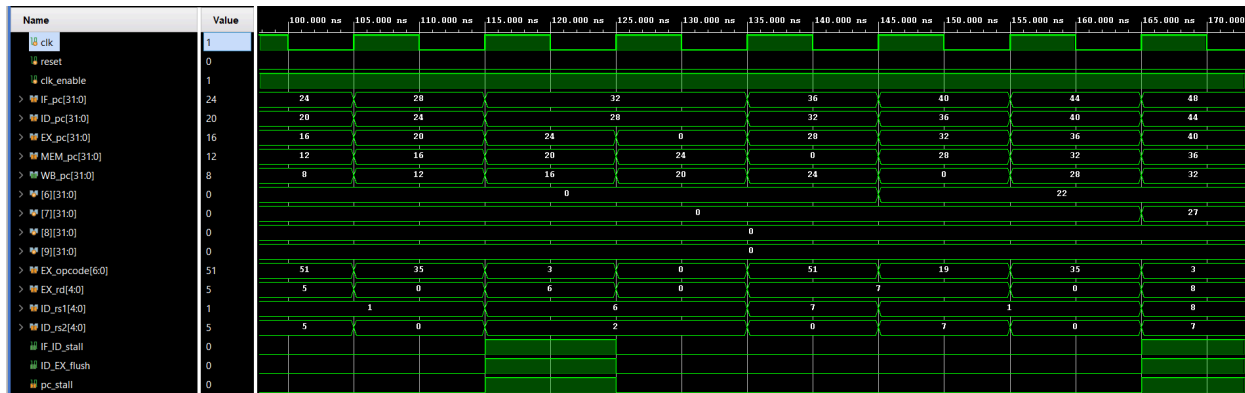
## Data Hazards load-use

When a Load-Use hazard occurs, forwarding is physically impossible because the memory data is not yet available. The Hazard Detection Unit must freeze the pipeline and insert a bubble.

The following sequence was executed to test simultaneous forwarding to both ALU inputs:

```
lw x6, 0(x1)    // EX Stage
add x7, x6, x2   // ID Stage (Needs x6, but it is currently loading)
```

| Signal Name | Binary Value | Description                                                   |
|-------------|--------------|---------------------------------------------------------------|
| EX_opcode   | 0000011      | Confirms the instruction in the EX stage is a <b>LOAD</b> .   |
| EX_rd       | 00110        | The <b>LOAD</b> instruction will write to x6.                 |
| ID_rs1      | 00110        | The <b>add</b> instruction in ID needs x6 (Matches EX_rd).    |
| ID_rs2      | 00010        | The <b>add</b> instruction in ID needs x2.                    |
| IF_ID_stall | 1            | Freezes the PC and IF/ID registers.                           |
| ID_EX_flush | 1            | Clears the control signals moving into EX to create a bubble. |



As shown in the waveform, the hazard is detected during the clock cycle where a load instruction (EX\_opcode = 3) is in the Execute stage with a destination of register x6 (EX\_rd = 6), while the dependent instruction in the Decode stage requires that exact same register as a source operand (ID\_rs1 = 6). Recognizing this unresolvable dependency, the Hazard Detection Unit immediately asserts its control signals (IF\_ID\_stall, ID\_EX\_flush, and pc\_stall all transition to high). Consequently, during the subsequent clock cycle, the Program Counter (IF\_pc = 32) and Decode stage (ID\_pc = 28) are frozen to hold their current instructions, and a NOP bubble (EX\_opcode = 0) is effectively forced into the Execute stage. This one-cycle stall successfully delays the dependent instruction until the memory read completes and the data becomes available for forwarding.

## Control hazards: Branch Not Taken

The following instructions were used to test Control Hazards:

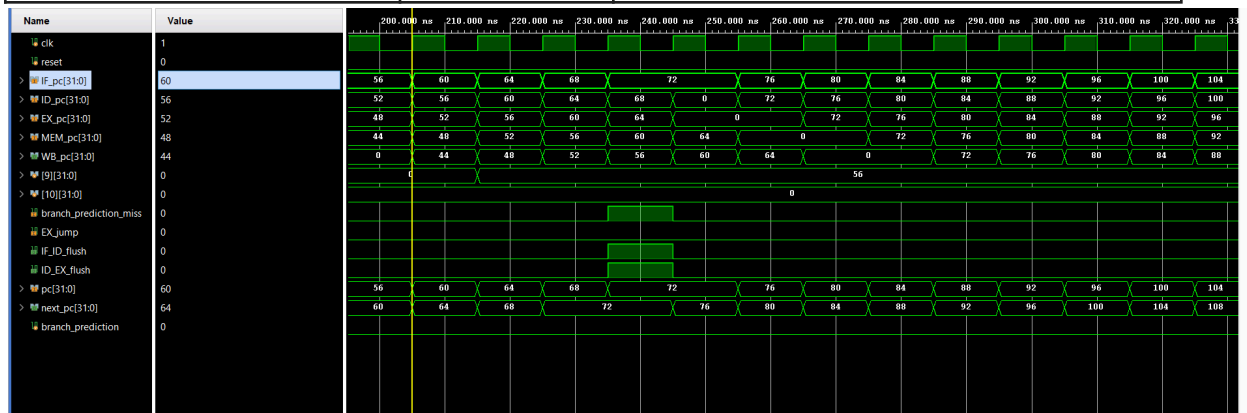
sub x10,x9,x9 // Set x10 =0

beq x10,x0,+8 // Branch to PC + 8 if x10 == 0

nop //skipped as branch is taken

| Signal Name       | Value         | Description                                      |
|-------------------|---------------|--------------------------------------------------|
| branch_prediction | 0 (Not Taken) | Predictor guessed the branch would fall through. |

|                        |            |                                                       |
|------------------------|------------|-------------------------------------------------------|
| branch_prediction_miss | 1          | Actual result was taken, so the prediction was wrong. |
| EX_jump                | 0          | This instruction is a branch, not a jump.             |
| IF_ID_flush            | 1          | Flushes the wrong-path instruction in IF/ID.          |
| ID_EX_flush            | 1          | Flushes the wrong-path instruction entering EX.       |
| pc                     | 0x00000034 | Address of data[13] (beq).                            |
| next_pc                | 0x0000003C | Branch target = pc + 8, so execution skips data[14].  |



In these set of instruction we show how the processor manages a branch prediction miss for a Branch at PC = 64. Initially, the branch predictor has predicted that a branch will not be taken. Thus, when the branch instruction reaches the EX stage, and a branch prediction miss is

detected, we need to flush the pipeline and start fetching the correct instructions. Thus, we observe that when EX\_pc = 64 (branch instruction), the IF\_ID\_flush and ID\_EX\_flush signals are asserted. Subsequently, the next\_pc is updated to 72 (right branch address).

### Control Hazards: JALR Data Dependency

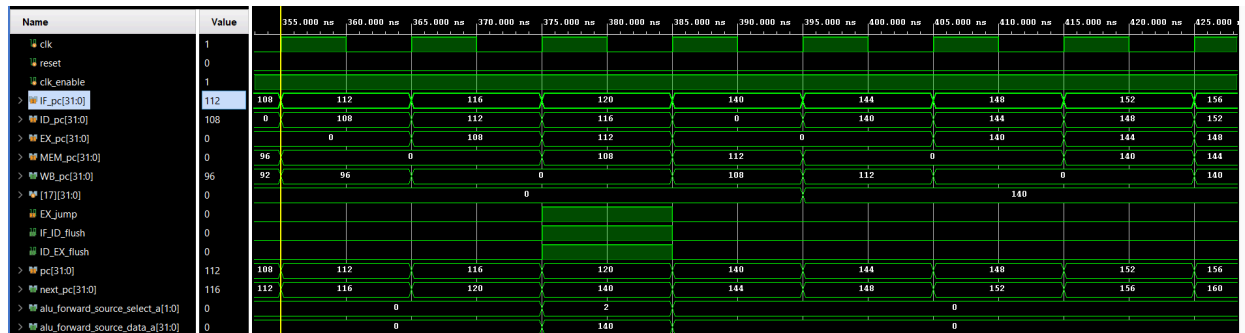
The following instructions were tested for JALR data dependency

```
addi x17, x0, 140
```

```
jalr x0, 0(x17)
```

```
// nop (skipped)
```

| Signal Name | Value      | Description                                          |
|-------------|------------|------------------------------------------------------|
| EX_jump     | 1          | Confirms the EX-stage instruction is a jump (jalr).  |
| IF_ID_flush | 1          | Flushes the younger wrong-path instruction in IF/ID. |
| ID_EX_flush | 1          | Inserts a bubble after the jump resolves.            |
| pc          | 0x00000070 | Address of data[28] (jalr).                          |
| next_pc     | 0x0000008C | Jump target comes from $x17 + 0 = 140 = 0x8C$ .      |



In these set of instructions, x17 is set to 140 and subsequently we jump to the address in the same register x17. This means that when the addi instruction is in WB stage, the jalr instruction would be in MEM stage having already completed execution, which would lead to a hazard as the correct value (140) has not arrived in the register x17. To solve this, the forwarding unit forwards 140 which is present in the EX\_MEM register, so that PC is updated to the right value. Since we have jumped to PC = 140, we would need to flush the IF\_ID and ID\_EX registers as they need to start fetching from PC = 140. Thus, the IF\_ID and ID\_EX signals are asserted and the pipeline begins execution from target address PC = 140.